

UNIDAD 3

MANIPULACIÓN DE ARCHIVOS Y MANEJO DE EXCEPCIONES

Tema:

ESTRUCTURA DE DATOS



UEA
UNIVERSIDAD
ESTATAL AMAZÓNICA

GUÍA DE ESTUDIO

SEMANA 9

ÍNDICE

1.1. INTRODUCCIÓN.....	1
1.2. RESULTADO DE APRENDIZAJE	2
1.3. RUTA DE APRENDIZAJE	2
1.4. DESARROLLO DE CONTENIDOS.....	3
1.4.1 Del objeto individual a la gestión de múltiples datos.....	3
1.4.2 Estructuras de datos integradas en Python	3
1.4.3 Principios SOLID: propósito y alcance	4
1.4.4 Listas: características, sintaxis y acceso	5
1.4.5 Operaciones básicas y CRUD con listas	6
1.4.6 Tuplas: datos ordenados e inmutables	7
1.4.7 Diccionarios: relaciones clave-valor	8
1.4.8 Conjuntos: valores únicos y pertenencia	10
1.4.9 Comparación de las cuatro estructuras	11
1.4.10 Estructuras de datos y objetos en POO	11
1.4.11 Integración con el proyecto Biblioteca	12
1.4.12 Fragmentos esenciales del código docente.....	13
1.4.13 Buenas prácticas aplicadas.....	16
1.4.14 Errores frecuentes y cómo interpretarlos.....	17
1.5. CONCLUSIONES.....	18
1.6. BIBLIOGRAFÍA.....	19

1.1. INTRODUCCIÓN

La Semana 9 inicia el estudio de las estructuras de datos integradas en Python dentro de la Unidad 3. Hasta este punto, los proyectos desarrollados durante el curso han permitido representar entidades individuales mediante clases y objetos. Un Libro representa un libro; un Usuario representa un usuario. Sin embargo, un sistema real necesita administrar varios objetos al mismo tiempo, conservarlos durante la ejecución, buscarlos, modificarlos y eliminarlos cuando corresponda.

Las estructuras de datos permiten organizar esos conglomerados de información de acuerdo con la necesidad del problema. Python ofrece, entre otras, listas, tuplas, diccionarios y conjuntos. Aunque todas permiten agrupar elementos, no poseen el mismo comportamiento: algunas son mutables, otras inmutables; algunas se consultan mediante índices, otras mediante claves; y los conjuntos se orientan a mantener elementos únicos (Python Software Foundation, 2026).

El caso de estudio continúa siendo un sistema de Biblioteca ejecutado mediante consola. La arquitectura académica ya conocida se conserva: los modelos representan las entidades, el servicio Biblioteca administra las operaciones y `main.py` coordina la interacción con el usuario. Esta semana no se incorpora una nueva capa; el cambio consiste en emplear las estructuras de datos de forma justificada dentro de los componentes existentes.

La guía desarrolla los conceptos de las diapositivas de la Semana 9 y los vincula con el código docente `biblioteca_app`. Se incluyen ejemplos breves de sintaxis y fragmentos esenciales del proyecto para observar cómo una lista administra objetos Libro y Usuario, cómo una tupla mantiene opciones estables del menú, cómo un diccionario relaciona claves con acciones o préstamos y cómo un conjunto permite obtener categorías sin duplicados.

1.2. RESULTADO DE APRENDIZAJE

Establece programas que manejan conglomerados de datos usando diferentes tipos de estructuras de datos, además del manejo de excepciones y colecciones.

1.3. RUTA DE APRENDIZAJE

La ruta de aprendizaje de la Semana 9 parte de una necesidad concreta: pasar de objetos individuales a la administración de múltiples registros. Primero se estudian las características de lista, tupla, diccionario y conjunto. Después se practican operaciones de acceso y modificación, con énfasis en las listas por su utilidad para administrar colecciones dinámicas. Finalmente, se revisa cómo estas estructuras se integran en el sistema Biblioteca sin alterar la arquitectura modular ya conocida.

COMPONENTE	ACTIVIDAD	CALIFICACIÓN
Componente en contacto con el docente	Cuestionario de comprensión sobre características, sintaxis, mutabilidad y selección de estructuras de datos.	10 puntos
Componente práctico-experimental	Taller de mejora progresiva del sistema Restaurante mediante estructuras de datos y operaciones básicas sobre registros.	10 puntos
Componente autónomo	Actividad de análisis y refuerzo para seleccionar estructuras adecuadas según distintos escenarios de programación.	10 puntos

Las actividades buscan que el estudiante relacione la sintaxis con una decisión de diseño concreta: qué datos necesita almacenar, cómo debe acceder a ellos y si deben modificarse, repetirse o mantenerse únicos.

1.4. DESARROLLO DE CONTENIDOS

1.4.1 Del objeto individual a la gestión de múltiples datos

En programación orientada a objetos, una clase define la estructura y el comportamiento de una entidad, mientras que cada objeto representa una instancia concreta. Esta representación resulta suficiente cuando se trabaja con un solo elemento; el problema aparece cuando el sistema debe administrar decenas o cientos de objetos.

```
libro_1 = Libro("L001", "Python", "Ana Pérez", "Programación")
libro_2 = Libro("L002", "POO", "Luis Torres", "Programación")
libro_3 = Libro("L003", "Algoritmos", "Marta Díaz", "Computación")
```

Mantener cada objeto en una variable independiente dificulta recorrerlos, buscarlos o aplicar una misma operación sobre todos. Una estructura de datos permite reunirlos bajo una colección administrable.

```
libros: list[Libro] = []

libros.append(libro_1)
libros.append(libro_2)
libros.append(libro_3)
```

A partir de ese momento, el programa puede recorrer la colección, consultar sus elementos, registrar nuevos objetos o eliminar registros existentes. Esta relación entre objetos y estructuras es especialmente importante en POO: la estructura de datos no sustituye al objeto; lo organiza junto con otros objetos del mismo sistema (Hernández Bejarano & Baquero Rey, 2025).

1.4.2 Estructuras de datos integradas en Python

Python incluye varias estructuras de datos de propósito general. En esta semana se estudian cuatro por su frecuencia de uso y por su relación directa con los proyectos del curso: list, tuple, dict y set.

- Lista (list): colección ordenada, mutable e indexada. Permite duplicados y resulta adecuada cuando los elementos pueden agregarse, modificarse o eliminarse.
- Tupla (tuple): secuencia ordenada e indexada, pero inmutable. Resulta adecuada para grupos de valores que deben permanecer estables.
- Diccionario (dict): estructura mutable de pares clave-valor. Cada clave es única y permite localizar directamente el valor asociado.
- Conjunto (set): colección mutable de elementos únicos. No ofrece acceso posicional mediante índices y resulta útil para eliminar duplicados o realizar operaciones de conjuntos.

La diferencia entre estas estructuras no es únicamente sintáctica. Su comportamiento determina qué operaciones son naturales y qué errores deben evitarse. Por ejemplo, una tupla no permite reemplazar un elemento una vez creada; un conjunto vacío debe construirse con `set()`, porque `{}` crea un diccionario vacío.

```
lista = ["Python", "P00"]
tupla = ("Python", "P00")
diccionario = {"codigo": "L001", "titulo": "Python"}
conjunto = {"Programación", "Computación"}

conjunto_vacio = set()    # Correcto
diccionario_vacio = {}    # También correcto, pero crea un dict
```

1.4.3 Principios SOLID: propósito y alcance

Seleccionar una estructura adecuada significa observar primero la necesidad de los datos. No se recomienda elegir list, tuple, dict o set por costumbre. La pregunta central es qué debe poder hacer el programa con esa información.

- Si la colección debe crecer, reducirse o modificar sus elementos, una lista suele ser una opción natural.
- Si el grupo de valores debe permanecer estable después de su creación, una tupla comunica mejor esa intención.
- Si cada dato debe localizarse mediante una clave significativa, un diccionario resulta más apropiado.

- Si lo importante es conservar valores únicos y evitar duplicados, un conjunto es la estructura indicada.

Estas decisiones pueden coexistir en un mismo sistema. Una aplicación no necesita elegir una única estructura para todo; puede usar varias estructuras, cada una en el lugar donde su comportamiento tenga sentido.

1.4.4 Listas: características, sintaxis y acceso

Una lista es una secuencia mutable. Los elementos mantienen un orden y cada posición puede consultarse mediante un índice que comienza en cero. Además, una lista puede contener valores repetidos y objetos de cualquier tipo. En proyectos orientados a objetos es frecuente declarar listas tipadas para dejar explícito qué clase de objetos se espera almacenar.

```
libros: list[Libro] = []
usuarios: list[Usuario] = []

libros = [
    Libro("L001", "Python", "Ana Pérez", "Programación"),
    Libro("L002", "P00", "Luis Torres", "Programación"),
]
```

El acceso posicional permite recuperar un elemento concreto cuando se conoce su índice.

```
primer_libro = libros[0]
print(primer_libro.titulo)

ultimo_libro = libros[-1]
print(ultimo_libro.titulo)
```

El índice 0 representa el primer elemento. Los índices negativos permiten acceder desde el final; por ejemplo, -1 corresponde al último elemento. Cuando el índice solicitado no existe, Python genera `IndexError`, por lo que el acceso posicional debe realizarse cuando exista certeza sobre el tamaño de la lista.

Las listas son especialmente útiles para los registros del sistema Biblioteca porque el número de libros y usuarios puede variar durante la ejecución.

1.4.5 Operaciones básicas y CRUD con listas

El término CRUD agrupa cuatro operaciones habituales sobre información: crear, consultar (read), actualizar y eliminar. En una lista estas acciones pueden relacionarse con la creación de la colección y la administración de sus elementos.

Crear y agregar:

```
libros: list[Libro] = []

libro = Libro("L001", "Python", "Ana Pérez", "Programación")
libros.append(libro)
```

append() agrega un elemento al final. insert(indice, elemento) permite insertarlo en una posición determinada.

Consultar:

```
print(libros[0])

for libro in libros:
    if libro.codigo == "L001":
        print(libro)
```

Consultar no implica únicamente usar un índice. En una colección de objetos es habitual recorrer la lista y comparar un atributo hasta localizar el registro buscado.

Actualizar:

```
for libro in libros:
    if libro.codigo == "L001":
        libro.titulo = "Python aplicado a P00"
        break
```

Actualizar una lista de objetos normalmente significa localizar el objeto y modificar sus propiedades. También es posible reemplazar directamente un elemento por índice, por ejemplo `libros[0] = nuevo_libro`.

Eliminar:

```
libros.remove(libro)
# o, cuando se conoce el índice:
libro_eliminado = libros.pop(0)
```

`remove()` elimina la primera coincidencia del valor indicado y produce `ValueError` si el elemento no se encuentra. `pop()` elimina y devuelve el elemento de una posición; si no se especifica un índice, elimina el último.

Herramientas complementarias:

```
cantidad = len(libros)
existe = libro in libros
```

`len()` devuelve la cantidad de elementos y el operador `in` permite comprobar pertenencia. Estas operaciones sencillas son suficientes para construir muchas funcionalidades de un sistema académico de consola.

Idea clave. En el proyecto docente, la lista no contiene diccionarios que sustituyan a *Libro* o *Usuario*; contiene objetos de esas clases, conservando las validaciones y responsabilidades propias de POO.

1.4.6 Tuplas: datos ordenados e inmutables

Una tupla es una secuencia ordenada e indexada cuyos elementos no pueden agregarse, eliminarse o reemplazarse después de crearla. Su inmutabilidad resulta útil cuando el programa necesita expresar que un grupo de valores forma parte de una configuración estable (Python Software Foundation, 2026).

```
coordenadas = (-1.8312, -78.1834)
print(coordenadas[0])

dia, mes, anio = (6, 8, 2026)
```

Una tupla con un solo elemento necesita una coma final. Sin ella, Python interpreta únicamente la expresión entre paréntesis.

```
tupla_un_elemento = ("Salir",)
no_es_tupla = ("Salir")
```

En el sistema Biblioteca, las opciones visibles del menú se definen mediante una tupla de pares. El usuario puede seleccionar una opción, pero la colección que describe el menú no necesita cambiar durante la ejecución.

```
OPCIONES_MENU = (
    ("1", "Registrar libro"),
    ("2", "Buscar libro"),
    ("3", "Actualizar libro"),
    ("0", "Salir"),
)
```

El uso de tuple comunica que se trata de una configuración estable y evita tratar el menú como una colección que deba crecer o reducirse mientras la aplicación está funcionando.

1.4.7 Diccionarios: relaciones clave-valor

Un diccionario almacena pares formados por una clave y un valor. Las claves deben ser únicas y de un tipo hashable, mientras que los valores pueden ser de distintos tipos. En las versiones modernas de Python, los diccionarios conservan el orden de inserción, aunque su característica esencial continúa siendo el acceso por clave.

```
libro = {
    "codigo": "L001",
    "titulo": "Python",
    "disponible": True,
}
```

```
print(libro["titulo"])
libro["autor"] = "Ana Pérez"
libro["titulo"] = "Python aplicado"
libro.pop("autor")
```

Cuando existe la posibilidad de que una clave no esté presente, `get()` permite consultar sin provocar `KeyError`.

```
autor = libro.get("autor")
autor = libro.get("autor", "Sin autor registrado")
```

Los métodos `keys()`, `values()` e `items()` permiten recorrer claves, valores o ambos. En POO, un diccionario no debe utilizarse automáticamente para reemplazar una clase cuando la entidad posee comportamiento y validaciones propias. En el proyecto docente se usa `dict` para relaciones y despachos por clave, no para sustituir `Libro` y `Usuario`.

Diccionario como mapa de acciones:

```
opciones = {
    "1": registrar_libro,
    "2": buscar_libro,
    "3": actualizar_libro,
}

accion = opciones.get(opcion)
if accion is not None:
    accion(biblioteca)
```

Aquí la clave representa la opción digitada por el usuario y el valor es la función que debe ejecutarse. Esto evita una cadena extensa de condiciones repetidas.

Diccionario para préstamos:

```
self._prestamos: dict[str, str] = {}

self._prestamos[libro.codigo] = usuario.identificacion
```

```
identificacion = self._prestamos.get(codigo_libro)
```

La relación código de libro -> identificación de usuario corresponde naturalmente a un diccionario: con una clave se obtiene el valor asociado.

1.4.8 Conjuntos: valores únicos y pertenencia

Un conjunto almacena elementos únicos. Si se intenta agregar un valor que ya existe, el conjunto no crea una segunda copia. No se accede por posición, de modo que expresiones como `conjunto[0]` no son válidas.

```
categorias: set[str] = set()

categorias.add("Programación")
categorias.add("Matemática")
categorias.add("Programación")

print(categorias)
```

Aunque se ejecuta `add('Programación')` dos veces, el valor aparece una sola vez. Para eliminar elementos pueden utilizarse `remove()` o `discard()`. `remove()` produce `KeyError` si el elemento no existe; `discard()` simplemente no realiza cambios.

```
categorias.discard("Historia")
existe = "Programación" in categorías
```

Los conjuntos también admiten operaciones matemáticas de unión, intersección y diferencia, útiles cuando se comparan grupos de valores.

```
a = {"Programación", "Matemática"}
b = {"Programación", "Historia"}

union = a | b
interseccion = a & b
diferencia = a - b
```

En Biblioteca se utiliza un conjunto para obtener las categorías existentes sin repetirlas, incluso si varios libros pertenecen a la misma categoría.

1.4.9 Comparación de las cuatro estructuras

La siguiente comparación resume las diferencias necesarias para seleccionar una estructura sin memorizar definiciones aisladas.

CARACTERÍSTICA	LISTA	TUPLA	DICCIONARIO	CONJUNTO
Sintaxis habitual	[]	()	{clave: valor}	{elementos} / set()
Mantiene orden de inserción	Sí	Sí	Sí	No debe usarse como secuencia posicional
Mutable	Sí	No	Sí	Sí
Duplicados	Sí	Sí	Claves: no / valores: sí	No
Acceso principal	Índice	Índice	Clave	Pertenencia / iteración

La tabla no debe interpretarse como una regla absoluta. Existen combinaciones y casos más avanzados, pero estos criterios son suficientes para seleccionar adecuadamente una estructura en los ejercicios de la asignatura.

1.4.10 Estructuras de datos y objetos en POO

Una estructura de datos y una clase cumplen propósitos diferentes. La clase modela una entidad con atributos, validaciones y comportamiento. La estructura administra varios elementos o establece una relación entre ellos.

```
class Libro:
    def __init__(self, codigo: str, titulo: str) -> None:
        self.codigo = codigo
        self.titulo = titulo

libros: list[Libro] = []
libros.append(Libro("L001", "Python"))
```

En este ejemplo, Libro continúa siendo responsable de representar cada libro, mientras que list se encarga de agrupar los objetos. Sustituir indiscriminadamente una clase por un diccionario puede hacer que las validaciones y comportamientos queden dispersos por el programa.

La separación entre modelos y colecciones favorece una organización más clara: los modelos representan; el servicio administra; main.py coordina la interacción. Esta distribución conserva la línea académica trabajada en las semanas anteriores y permite que el proyecto evolucione gradualmente (Boucheny, 2025).

1.4.11 Integración con el proyecto Biblioteca

La arquitectura del proyecto no cambia en la Semana 9. Se reutiliza la organización modular ya conocida y se incorporan estructuras de datos dentro de los componentes donde existe una necesidad concreta.

```
biblioteca_app/  
├─ modelos/  
│   ├─ __init__.py  
│   ├─ libro.py  
│   └─ usuario.py  
├─ servicios/  
│   ├─ __init__.py  
│   └─ biblioteca.py  
├─ main.py  
└─ README.md
```

El servicio Biblioteca mantiene dos listas mutables porque durante la ejecución se registran y eliminan objetos. También utiliza un diccionario para asociar cada libro prestado con un usuario. El menú utiliza una tupla para conservar opciones estables, y las categorías únicas se obtienen mediante un conjunto.

Idea clave. La Semana 9 mejora el manejo de datos dentro de la arquitectura existente; no introduce carpetas nuevas ni modifica la separación de responsabilidades.

1.4.12 Fragmentos esenciales del código docente

Los siguientes fragmentos se han tomado del proyecto docente biblioteca_app de la Semana 9. Se seleccionan únicamente las partes necesarias para reconocer el uso justificado de list, tuple, dict y set. El proyecto completo debe revisarse y ejecutarse desde el repositorio docente.

Repositorio docente: <https://github.com/kevin10lascano-sketch/Clase-Semana-9-POO.git>

a) Colecciones principales del servicio:

```
class Biblioteca:
    def __init__(self) -> None:
        self._libros: list[Libro] = []
        self._usuarios: list[Usuario] = []
        self._prestamos: dict[str, str] = {}
```

Las listas almacenan objetos Libro y Usuario porque ambas colecciones cambian durante la ejecución. El diccionario _prestamos expresa una relación directa entre el código de un libro y la identificación del usuario que lo posee.

Repositorio docente:

b) Registro, búsqueda, actualización y eliminación sobre una lista:

```
def registrar_libro(self, libro: Libro) -> bool:
    if self.buscar_libro(libro.codigo) is not None:
        return False

    self._libros.append(libro)
    return True

def buscar_libro(self, codigo: str) -> Libro | None:
    codigo = codigo.strip()
    for libro in self._libros:
```

```
        if libro.codigo == codigo:
            return libro
    return None
```

registrar_libro() utiliza append() después de controlar duplicados por código.

buscar_libro() recorre la lista y devuelve el objeto encontrado. Esta búsqueda se reutiliza luego para actualizar o eliminar, evitando repetir la misma lógica.

```
def actualizar_libro(
    self,
    codigo: str,
    nuevo_titulo: str,
    nuevo_autor: str,
    nueva_categoria: str,
) -> bool:
    libro = self.buscar_libro(codigo)
    if libro is None:
        return False

    libro.titulo = nuevo_titulo
    libro.autor = nuevo_autor
    libro.categoria = nueva_categoria
    return True

def eliminar_libro(self, codigo: str) -> bool:
    libro = self.buscar_libro(codigo)
    if libro is None:
        return False
    if codigo.strip() in self._prestamos:
        return False

    self._libros.remove(libro)
    return True
```

La actualización modifica propiedades del objeto almacenado; no reemplaza necesariamente toda la posición de la lista. La eliminación utiliza remove() y, antes de ejecutarse, comprueba que el libro no se encuentre asociado a un préstamo.

c) Copia de la lista al exponer resultados:

```
def listar_libros(self) -> list[Libro]:  
    return self._libros.copy()
```

Devolver `copy()` evita entregar directamente la lista interna del servicio. El código que recibe el resultado puede recorrerla sin obtener una referencia para agregar o eliminar elementos de `_libros` accidentalmente. Es una práctica sencilla de encapsulación aplicada a colecciones.

d) Tupla para opciones estables del menú:

```
OPCIONES_MENU = (  
    ("1", "Registrar libro"),  
    ("2", "Buscar libro"),  
    ("3", "Actualizar libro"),  
    ("4", "Eliminar libro"),  
    ("5", "Listar libros"),  
    ("0", "Salir"),  
)  
  
def mostrar_menu() -> None:  
    for numero, descripcion in OPCIONES_MENU:  
        print(f"{numero}. {descripcion}")
```

La tupla representa una colección definida por el programa. El usuario selecciona opciones, pero no modifica la configuración del menú.

e) Diccionario para despachar acciones:

```
opciones = {  
    "1": registrar_libro,  
    "2": buscar_libro,  
    "3": actualizar_libro,  
    "4": eliminar_libro,  
    "5": listar_libros,  
}
```

```

accion = opciones.get(opcion)
if accion is None:
    print("Opcion invalida.")
else:
    accion(biblioteca)

```

Cada clave del diccionario se asocia con una función. Esta organización evita una cadena extensa de if/elif y muestra un uso práctico de clave -> valor dentro del menú.

f) Conjunto para categorías únicas:

```

def obtener_categorias_unicas(self) -> set[str]:
    categorias: set[str] = set()
    for libro in self._libros:
        categorias.add(libro.categoria)
    return categorias

```

Si varios libros pertenecen a la misma categoría, add() no genera duplicados. El conjunto expresa directamente la intención de obtener valores únicos.

```

for categoria in sorted(categorias):
    print(f"- {categoria}")

```

Cuando se desea presentar el conjunto ordenado al usuario, sorted() produce una secuencia ordenada para la salida sin convertir el conjunto original en una estructura posicional permanente.

1.4.13 Buenas prácticas aplicadas

El uso de estructuras de datos debe mantener la claridad del diseño. Las siguientes prácticas son especialmente útiles en el alcance de esta semana.

- Utilizar anotaciones de tipo, por ejemplo list[Libro], dict[str, str] y set[str], para comunicar qué elementos espera cada colección.
- Mantener las colecciones dentro del servicio cuando forman parte de la lógica de administración del sistema.

- Evitar que main.py modifique directamente las listas internas del servicio; main.py debe solicitar operaciones mediante métodos.
- Controlar duplicados con un identificador significativo antes de registrar un nuevo objeto.
- Devolver copias de listas cuando no se desea exponer la colección interna para modificación directa.
- Usar tuple cuando la colección es estable por intención, no únicamente porque su sintaxis sea corta.
- Usar dict cuando existe una relación clara clave -> valor; no convertir todas las entidades del dominio en diccionarios.
- Usar set cuando la unicidad sea parte real del problema.
- Elegir discard() cuando eliminar un valor inexistente no deba considerarse un error; utilizar remove() cuando la ausencia sí deba detectarse.
- Mantener validaciones en los modelos o servicios correspondientes y capturar errores de entrada en la interacción con el usuario.

Idea clave. Una estructura de datos bien elegida hace visible la intención del código y reduce operaciones innecesarias.

1.4.14 Errores frecuentes y cómo interpretarlos

Las primeras prácticas con estructuras de datos suelen producir errores previsibles. Comprenderlos ayuda a diagnosticar el programa en lugar de limitarse a copiar métodos.

IndexError en listas y tuplas:

```
libros = []
print(libros[0]) # IndexError
```

El índice solicitado no existe. Antes de acceder por posición debe comprobarse que la colección contenga elementos o utilizar una búsqueda apropiada.

KeyError en diccionarios:

```
datos = {"codigo": "L001"}
print(datos["autor"]) # KeyError
```

La clave no existe. Si su presencia es opcional, puede utilizarse get().

TypeError al modificar una tupla:

```
opciones = ("Registrar", "Buscar")  
opciones[0] = "Nuevo" # TypeError
```

La tupla es inmutable. Si los elementos realmente necesitan cambiar, debe revisarse si list sería una elección más apropiada.

Confundir un conjunto vacío con un diccionario:

```
categorias = {} # dict  
categorias = set() # set
```

Las llaves vacías representan un diccionario. set() es la forma correcta de crear un conjunto vacío.

Eliminar valores inexistentes:

```
categorias = {"Programación"}  
categorias.remove("Historia") # KeyError  
categorias.discard("Historia") # No genera error
```

La elección entre remove() y discard() depende de si la ausencia del elemento debe ser tratada como una condición excepcional.

1.5. CONCLUSIONES

Las estructuras de datos permiten organizar y administrar múltiples elementos dentro de un programa. En Python, las listas, tuplas, diccionarios y conjuntos ofrecen comportamientos diferentes que deben elegirse según la forma en que los datos necesitan almacenarse, consultarse y modificarse.

Las listas resultan adecuadas para colecciones dinámicas y permiten realizar operaciones de registro, búsqueda, actualización y eliminación. Las tuplas representan secuencias estables e inmutables. Los diccionarios modelan asociaciones mediante claves únicas y los conjuntos resuelven problemas en los que la unicidad de los valores es importante.

En programación orientada a objetos, estas estructuras complementan a las clases: los objetos representan entidades y las estructuras permiten administrar grupos o relaciones entre esos objetos. Por ello, el proyecto Biblioteca conserva sus modelos, servicios y archivo principal, e incorpora list, tuple, dict y set en los lugares donde cada estructura aporta un comportamiento concreto.

La selección adecuada de una estructura mejora la claridad del código y prepara al estudiante para trabajar posteriormente con colecciones de objetos, búsquedas, filtrados y transformaciones de información.

1.6. BIBLIOGRAFÍA

Boucheny, V. (2025). Aprender programación orientada a objetos con Python (3.^a ed.). Ediciones ENI.

Downey, A. (2024). Think Python (3rd ed.). O'Reilly Media.

Hernández Bejarano, M., & Baquero Rey, L. E. (2023). Programación orientada a objetos en Java: Buenas prácticas. Ediciones de la U.

Hernández Bejarano, M., & Baquero Rey, L. E. (2025). Python con orientación a objetos y al análisis de datos. Ediciones de la U.

Piñeiro Gómez, J. M. (2022). Principios de la programación orientada a objetos. Paraninfo.